1BM110 - Assignment 2

Deep Reinforcement Learning

Jesse van Remmerden, Dr. Zaharah Bukhsh

2025-2026

1 Problem Description

The Bounded Knapsack problem is a combinatorial optimization problem. It requires the user to select from a range of goods of different values and weights to maximize the value of the selected items within a given weight limit. The bounded implies that each item can be selected a limited number of times. *For this assignment, you will train a reinforcement learning agent to solve the Bounded Knapsack problem with 200 items.*

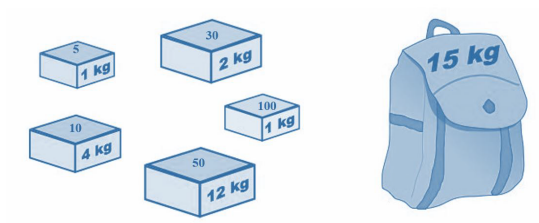

Figure 1: Illustration of Knapsack problem

The formalization of the bounded knapsack problem is as follows:

$$\begin{aligned} \text{maximize} \sum_{i=1}^{n} p_i x_i \\ \text{subject to} \sum_{i=1}^{n} w_i x_i \leq W, \\ 0 \leq x_i \leq c_i, \quad x_i \in \mathbb{Z} \end{aligned}$$

Whereby n the number of items in the problem, p_i the value is of items value, x_i the amount of item i in the knapsack, w_i the weight of item i, and W the maximum capacity of the knapsack.

The environment is adapted from the OR-GYM [1] library and is based on the Gymnasium framework. The **RL model's state** is defined as a concatenation of vectors containing items weight, items value, items limit, as well as the knapsack's current load and maximum capacity. The number of actions is equal to the number of items available. The **reward** is the total value of all items placed within the knapsack. In the environment, we have multiplied the **reward** with 0.01, since lower values are more easily to learn for neural networks. This entails that if the **total reward** is 10, the real value is 1000. At each timestep, the agent can select only one item. An episode ends when the knapsack is full or no additional items can be placed.

2 The assignment

The assignment has three parts. For each part, we first explain the assignment and afterwards the work structure you should use for this part.

2.1 Part 1: The environment

Using the problem description in Section 1 and the provided environment Python file, describe the Markov Decision Process (MDP) of the bounded knapsack problem.

Tasks

- Begin with an overview of the `BoundedKnapsack` environment and how it relates to the MDP framework.
- Provide a detailed explanation of how an RL agent interact with this environment.

2.2 Part 2: Training the Agent

Using the algorithm implementations provided in the **Stable Baselines3** package, train a Reinforcement Learning agent that uses a neural network as a function approximator to solve the **Bounded Knapsack** environment. You will train both a Deep Q-Network (DQN) agent [2] and a Proximal Policy Optimization (PPO) agent [3]

Tasks

- Start by explaining both DQN [2] and PPO [3] in the report. Describe essential aspects, such as how each method updates their weights and when they do their update.

- Implement both DQN [2] and PPO [3], found in Stable Baselines 3 [4]. Train both methods for $50,000$ training steps with 3 different seeds (use the same 3 seeds for both algorithms), evaluate the trained agents, and plot the training graphs.
  - When reporting results across the 3 seeds, do not plot each seed individually. Instead, aggregate the results. For example, by reporting mean, also ensure that the standard deviation of the seeds 3 is conveyed.
- Next, select two hyperparameters that both PPO and DQN use and two hyperparameters specific to each method. Tune all four by hand, testing at least 5 different values per hyperparameter using the same three seeds as before.
  - Ensure that you detail the tuning in the report. We recommend using visualizations to convey this information; however, it is up to you how to do this.
  - **Important:** Not all hyperparameters are equally important for agent's performance. It is your task to select four of the most important hyperparameters and their values. Motivate the choices made in the report.
- Compare the performance of DQN and PPO, and explain any differences observed in their results.

2.3 Part 3: Training the Agent with Invalid Action Masking

In the previous part, your agents learned to exploit the information contained in the environment state to decide which item to pack next. Since an episode ends every time the agent selects an item that does not fit the knapsack or is no longer available, the neural network must implicitly learn to avoid selecting those items. However, given the state of the environment at every step it is easy to compute whether an item (or action) can or cannot still be selected. The BoundedKnapsack environment offers an action mask that is automatically updated after every step and can be accessed through the method `env.get_mask()`.

This information can be used in policy gradient algorithms to:

- prevent the agent from selecting invalid actions during inference
- improve training by explicitly signaling which actions are invalid in a given state, allowing the agent to focus on learning the value of valid actions.

In particular, the process of action masking for policy gradient algorithms (see Figure 2 consists of setting the logits (obtained from the output layer of the policy network) to minus infinity (or the lowest possible value) for the values

corresponding to the invalid actions such that the related probability will be 0 after applying the softmax function.

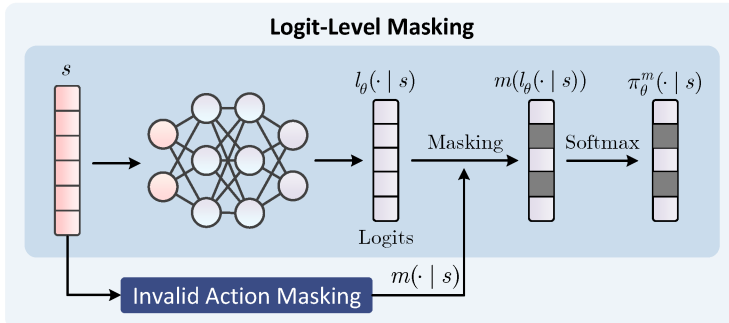

Figure 2: Overview of logit-level masking

Using the algorithm implementation provided in the **SB3 Contrib** package, train a MaskablePPO agent.

Tasks

- Enable action masking, and train and evaluate a `MaskablePPO` agent from SB3 Contrib [5] on the `BoundedKnapsack` environment using the same 3 seeds and $50,000$ training steps as in Part 2.
- Use the same hyperparameters as in Part 2 for PPO, then explore new values (atleast three) to see whether performance can be improved further. Again, use the same three seeds.
- Evaluate the agent and compare the best results obtained against those of PPO from Part 2.

Hints

- Start by looking at the description of the Bounded Knapsack environment to understand how the RL agent learns.
- For visualising your results, take inspiration from the DQN [2] and PPO [3] articles to see how results are typically reported. You have multiple ways of visualizing results, so there is not a single way of doing it. However, it is your job to ensure that the information is clearly communicated to the reader.
- You need to run PPO, DQN and MaskedablePPO multiple times with three different seeds, so spent some time at the start to create an experimental setup to easily run each method. By doing this correctly, you can save yourself a lot of time and make the assignment much easier.

3 Submission details

The project must be implemented using Python 3.11 and PyTorch 2.0+. The submission folder of the project should contain the following files:

- Python file, containing:
  - Code (well-commented) for training and testing for the DRL agents;
- Report PDF file that answers tasks noted in Section 2, containing
  - A description of the environment;
  - Motivation and discussion of hyperparameter choices and their impact on performance;
  - Visualization and discussion of results for each agent;
  - The best reward achieved and total training steps for each agent;
  - A comparison of agent performances with an explanation of the observed differences.

The files described must be submitted in a zip file containing one single directory, as shown in Figure 3. The naming convention for the directory is "Group-number_1BM110_A2" (e.g. Group00_1BM110_A2). The maximum length of the report is **6 pages**, excluding a title page and optional references.

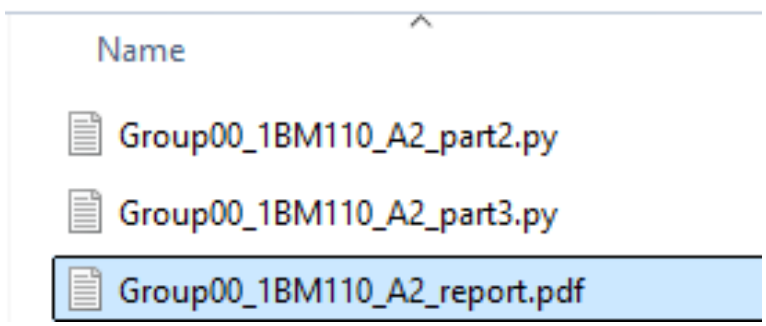

Figure 3: Guide for files names and submission directory structure

4 Tips for getting started

1. To install Gymnasium, simply use pip:

   ```
   pip install gymnasium
   ```

2. You can set the seed as follow for the environment (It is your task to do this always correctly and multiple times):

```
import numpy as np
seed = (number of your choice, e.g. 2024)
np.random.seed(seed)
```

3. You can set the seed as follow for the agent, for DQN and PPO it is the same. The documentation of Stable Baselines explains it in more detail.:

```
from stable_baselines3 import DQN
model = DQN(..., seed=seed)
```

4. Download the provided knapsack_env.py file. Import the environment from the provided file:

```
from knapsack_env import BoundedKnapsackEnv
```

5. Create an instance of the environment with the following parameters:

```
env = BoundedKnapsackEnv(n_items=200, max_weight=200)
```

   (a) For Part 3, you will first need to enable the mask:
   `env = BoundedKnapsackEnv(n_items=200, max_weight=200, mask=True)`

   (b) Then, you need to wrap the environment with the ActionMasker:
   `env = ActionMasker(env, mask_fn)`

   Note that the implementation of the `mask_fn` function is left to you.

6. Inspect the state space and action spaces as follows:

```
state_space = env.reset()
action_space_size = env.action_space.n
```

7. To install Stable Baselines3 and SB3 Contrib use pip:

```
pip install stable-baselines3
pip install sb3-contrib
```

5 References

[1] Christian D. Hubbs, Hector D. Perez, Owais Sarwar, Nikolaos V. Sahinidis, Ignacio E. Grossmann, and John M. Wassick. Or-gym: A reinforcement learning library for operations research problems, 2020.

[2] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin A. Riedmiller, Andreas Fidjeland, Georg Ostrovski, Stig Petersen, Charlie Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

[4] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[5] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Sb3 contrib. `https://github.com/Stable-Baselines-Team/stable-baselines3-contrib`.